ECE 2049 Reimagined MQP

Project Report

Kyle Schmottlach and Shannon Miranda

December 14, 2025

Authorship Table

Section	**Original Author(s)**	**Editor(s)**
1 Introduction	Kyle	
2 Literature Review	Kyle	Shannon
2.1 Embedded Systems	Kyle	
2.1.1 ESP32	Kyle	
2.1.2 STM32	Kyle	
2.2 Importance of Education	Kyle	
2.3 WPI ECE 2049 Course	Kyle	Shannon
3 Methodology	Shannon	Kyle
3.1 Interviewing Schools	Shannon	Kyle
3.2 Board Testing		
4 Preliminary Findings	Kyle & Shannon	
4.1 Board Testing	Shannon	Kyle
4.2 EFR xG26-DK2608A	Shannon	
4.3 ESP32-C6-DevKitC-1	Kyle	
4.4 STM32 Nucleo	Kyle	
4.4.1 STM32CubeIDE - Eclipse	Kyle	
4.4.2 STM32CubeIDE - VSCode	Kyle	
4.4.3 STM Nucleo-G491RE	Shannon	Kyle
4.4.4 STM Nucleo-H533RE	Kyle	
4.5 Board Choice	Kyle	

1 Introduction - Kyle

Throughout the past 50 years, embedded systems have accrued a rich history. Embedded systems are devices that have a tight relationship between their hardware and software, often performing a single function as a part of a larger system [1]. They often must operate in real-time such that data processing does not exceed the rate of data input, to adequately complete their functions within the larger system [1]. During their infancy in the early 1960s, they followed a mostly proprietary paradigm, with the first being built specifically for NASA spaceships and satellites during the Apollo Era [1]. Embedded systems did not take off in popularity until the introduction of the microprocessor (a single chip acting as a small CPU), spearheaded by Texas Instruments (TI) and Intel, with another proprietary implementation invented by the US Navy in the 1970s [1]. The microprocessor, when combined with memory units such as RAM and ROM and Input/Output (I/O) interfaces on a single integrated circuit, creates the Microcontroller Unit (MCU), the basis of many embedded systems today [2].

Embedded systems play a crucial role in the proper functioning of devices that society uses every day, ranging from consumer electronics to vehicles to military operations to the medical fields. Consumer electronics such as audio and video devices (like smart TVs) integrate embedded systems to enable more complex processing and user experiences [3]. Smartphones, smart watches, game consoles, and even household appliances like washing machines and dishwashers all integrate embedded systems for specific, tailored computing use cases, more specifically enabling Internet of Things devices to function correctly in low-power, specific use-case environments. Modern cars integrate embedded systems for the proper operation of the engine, safety systems, autonomous systems, and GPS. Airplanes also utilize this technology to facilitate autonomous piloting, anti-collision technologies, and information systems. On the military side, embedded systems are critical to the proper functioning of drones, fire control systems, and tracking devices. Finally, within the medical sectors, modern medical equipment relies on the functionality of embedded systems to create smart health tracking devices, such as internal defibrillators, digital x-rays, electronic scales, and stat monitoring machines [3]. In the modern age, embedded systems exist in the majority of sectors of daily life, and their popularity only continues to increase.

As technology has grown exponentially, their small form factor, lightweight nature, specific designs, low cost, and low power features, among others, have also contributed to a surge in their use across several fields of computing, including networking, mobile computing, and the industrial complex [2]. In 2024, the North American embedded system market held a market size of roughly $112.3 billion, holding a projected market size of $169.1 billion by 2030, with the healthcare market representing an area of vast projected growth [4]. These increases in the use of embedded systems across these different fields and the vast expected increase in the market size of embedded systems increase demand for embedded programmers and designers, providing job security [3] and opportunities for interdisciplinary study regarding embedded systems. As such, complete and modern education in embedded systems is critical to support this growing market.

At WPI, the ECE 2049 course is the primary introductory course to embedded systems. Started in 2012, the course teaches fundamental concepts of embedded systems, encompassing both hardware and software aspects, to introduce students to this field. The course is heavily project-based, gradually building up practical knowledge of theoretical topics from lectures as the term progresses. The course currently uses the Texas Instruments (TI) MSP430F5529 (MSP430) LaunchPad with the BoosterPack extension board, providing a wide variety of peripherals that students can interact with. However, recent updates to the TI integrated development environment (IDE), Code Composer Studio (CCS), have switched the backend from Eclipse to Theia (a TI clone of Microsoft's Visual Studio Code). This switch has significantly altered the UI for the IDE and removed or hidden many features that are used extensively in the ECE 2049 course.

As such, this project aims to modernize the WPI ECE 2049 course by integrating new embedded technology, updating course materials, and realigning course content with industry standards. We plan to accomplish this goal via three objectives. Firstly, we will understand industry standards and current trends in the education sector on embedded systems, and courses that teach the field, to shape the ECE 2049 requirements. Secondly, we will determine and form an ideal embedded systems lab kit based on those new course requirements. Finally, we will gather and understand the ideal components of lecture materials, lab activities, and homework assignments.

2 Literature Review - Kyle

2.1 Embedded Systems

Today, embedded systems form the backbone of many computing and Internet of Things (IoT) devices, enabling tailored computing across the globe for small devices specifically designed for use cases in homes, cars, and appliances [5]. Embedded systems continue to remain at the center of the technology boom across the computing world, and their relevance and indispensability to today's computerized environment are ever-increasing. Several different families of MCUs exist on the market that cater to specific features and use cases, developed as these three fields have grown.

2.1.1 ESP32

The ESP32 line is a family of MCUs created by Espressif Systems, designed for mobile, wearable, and IoT computing applications [6]. The ESP32 family has a wide variety of MCU product lines, each with different capabilities: the ESP32-P high-performance series offers dual-core and image and video processing capabilities; the ESP32-S2 and ESP32-S3 series offer wireless functionality tailored for cloud computing, with the S3 line offering a dual-core MPU supporting vector instructions that enable neural network acceleration; the more general-purpose ESP32-C and ESP-H series provide varying support for wireless and peripheral technologies [7].

2.1.2 STM32

The STM32 family encompasses an extensive range of MCUs, including options for high-performance, mainstream, ultra-low-power, and wireless use cases [8]. These MCUs have a wide variety of on-board peripherals that support connections to many different sensors and control systems [6]. STMicroelectronics additionally provides an extensive set of prototyping boards through its Nucleo line. Each STM32 Nucleo board contains a specific MCU along with a set of connectors exposing the I/O pins built into the MCU [9]. These pins are arranged such that many ARDUINO® Uno V3 and ST morpho shields can be seamlessly connected into the board and integrated into an embedded project, allowing for wide support for external interfaces [9]. Most Nucleo boards also contain STMicroelectronics' proprietary STLink debugger/programming interface that connects the board directly to software debuggers offered through their free IDE software [9].

2.1.3 EFR32

TBD

2.2 Importance of Education

The unfettered growth of embedded systems resulting from the global expansion of technology has increased the demand for clear and updated education on embedded systems, both in theory and in practice [10]. At the intersection of computer science, electrical engineering, and computer engineering, embedded systems represent a highly interdisciplinary field of study [10], making modernizations to course teachings necessary to reflect the constant innovation that occurs within these fields [11]. The Association for Computing Machinery and the IEEE Computer Society published a report outlining general guidelines for the computer engineering curriculum in 2016. In the report, the authors describe how many computer engineers in the field of embedded system design often create computer-based systems for specific and specialized applications. As technology has evolved, the report acknowledges that various technologies, including multimedia, computers, and networking, are converging within the computer engineering field, which in turn increases demand in the field of computer engineering [11].

The report also outlines several expected characteristics that undergraduate computer engineering students should have: professionalism, ability to design, breadth of knowledge, and preparation for professional practice [11]. Computer engineering systems have both direct and indirect effects on public society. As such, professionalism in the design and implementation of computer systems is important to ensure compliance with societal and ethical requirements. When creating new computing systems, having the ability to design is critical for ensuring continued advancement in the field. According to the report, students in computer

engineering should be well-versed in the stages of the engineering design process and be able to integrate new theories as well as scientific and mathematical principles into new hardware, software, networks, and processes to advance the field. Furthermore, the report introduces the importance of having a specific set of common topics for all computer engineering programs, in addition to any topics that the course wishes to explore in depth. These topics include system-level perspectives, depth and breadth, design experience, use of various laboratory tools, professional practice, and written, oral, and graphical communication skills [11].

2.3 WPI ECE 2049 Course

The ECE 2049: Embedded Computing in Engineering Design course at WPI has used the same MSP430-F5529LP Launchpad (containing the MSP430F5529 MCU) since approximately 2018. While the MSP430 offers an impressive set of features, including a full-speed USB bus, low-power modes, a 12-bit ADC, 3-channel DMA, four 16-bit timers, and two Universal Serial Communication Interfaces supporting the UART, SPI, and I2C communication interfaces [12], newer MCUs and their respective development boards, such as those from the ESP32, STM32, and EFR32 families, offer a wider range of peripherals as well as upgraded sets of the peripherals that exist on the MSP430. Additionally, the MSP430 only supports extension or booster packs that come directly from TI [12], limiting the options for expansion outside of TI-provided systems. With many development boards from other families, especially the STM32, shield boards with the Arduino connector format are supported, allowing any supported Arduino shield to be used within the MCU environment. Furthermore, newer technologies such as dual-core processing and neural network/machine learning acceleration are not available on the MSP430. Since the ECE 2049 course prepares ECE students at WPI for the current trends in embedded systems, the MSP430 has reached the end of its functional lifetime within the course, and an upgrade is necessary.

3 Methodology - Kyle

3.1 Introduction

In this project, we plan to modernize the ECE 2049 Embedded Computing in Engineering Design course at WPI by integrating new embedded technology, updating lab materials, and aligning course content with industry standards. To achieve this goal, we aim to complete three main objectives: evaluate the current industry standards and education trends in embedded systems, develop a modern embedded systems lab kit integrating current industry and educational standards, and modernize current lab demonstrations and assignments to align with the new lab kitmodernize current lab demonstrations and assignments to align with the new lab kit. In this section, we break down each project objective and its relevance to the overarching project goal, and we describe the specific methods used to achieve each objective.

3.2 Objective 1: Evaluate the current industry standards and education trends in embedded systems - Shannon

Every year, the U.S. Department of Education's National Center for Education Statistics (NCES) conducts a series of 12 interrelated surveys known as the Integrated Postsecondary Education Data System (IPEDS) [13]. All institutions with federal student aid programs are required to fill out these surveys under 20 U.S. Code § 1094 Section 487(a)(17) and 34 CFR 668.14(b)(19). These surveys cover 8 areas including institutional characteristics, institutional prices, admissions, enrollment, student financial aid, degree/certificate completions, student persistence/success, and institutional resources [13]. One of the questions asks the institute to select which they believe to be their peer schools. There are a multitude of reasons why an institution may choose another. It could be one they believe is similar, one the institution desires to be like, or one that holds the same values. WPI selected 12 schools as its peers, and half of those selected it back. This is where we began to collect a list of potential schools to investigate.

In order to determine what constitutes a good introductory embedded systems course, we can look at what other schools are doing. We need to know what works and what doesn't work with the materials they have. By investigating multiple schools, we can gauge opinions on comparable options and determine what the current trends are in the industry.

We compiled a list of 14 institutions that we believed would provide a good comparison. We made a conscious effort to include a diverse mix of size, type, and reputation. Then we looked at the publicly

available course catalogs and found the synopsis of each institution's introductory embedded systems course. Some schools split their classes differently or lean extra heavily towards either the hardware or software side, so we removed them from our list. Ultimately, we identified six schools that offered a class similar to our ECE 2049 and shared a learning philosophy similar to WPI. One of these schools, Clarkson University, was a mutually chosen peer school as well. We also contacted Stevens Institute of Technology, Rowan University, Virginia Tech, Rose-Hulman Institute of Technology, and Rensselaer Polytechnic Institute. We received responses from Clarkson, Rowan, and Virginia Tech, who were all more than willing to help us however they could. We received syllabi and various other course materials from the professors about their respective classes. We were also able to conduct an interview with each of them.

Learning genuine information was essential for us, so we prefaced each interview with a promise of anonymity. We did not record the interviews or use any of Zoom's AI transcribers/summarizers. Our plan for the interviews was to have Kyle moderate and work through our list of questions while Shannon took notes on everything. However, the professors were very enthusiastic and mostly shared all the info we needed to know without being prompted by our questions. They offered insight not only on the microcontrollers they utilized, but also on course structure and feasibility. One university professor invited the two staff engineers to our meeting as well. Their different points of view provided much-needed knowledge and reminded us to think about what projects/labs we can do with the board and how this can be applied to industry/future applications/advanced projects

3.3 Objective 2: Develop a modern embedded systems lab kit integrating current industry and educational standards - Kyle

Objective 2 describes the culmination of learned industry and educational standards from Objective 1 with background research on the individual components, functionality, and quality of documentation of individual microcontroller boards and their associated Development Kits (Dev Kit). This objective operates chronologically between Objectives 1 and 3, requiring the context from Objective 1 of prior research and opinions from individuals in the field to make informed decisions on adequate and promising microcontroller boards while informing the changes that should be made to the lab demonstrations and assignments for Objective 3. To perform proper and comparable testing between each of the boards, we formed a set of testing metrics that were standardized across all microcontrollers in our test set. This standardization will allow us to fairly compare each board with all others in the testing set and draw informed conclusions about which boards are objectively better fits for the course.

3.3.1 Forming the Board Testing Set

Using the insights we gained from Objective 1 and our background research, we developed pre-testing criteria to compare a large set of microcontroller units. To help guide our decisions, we chose to compare microcontrollers from three popular microcontroller development companies: Silicon Labs, STMicroelectronics (STM), and Espressif Systems. These companies were chosen for their wide selection of microcontrollers and their associated Dev Kits that are still within support, as well as their clear classification of microcontrollers into families based on their feature sets. From the most recent families for each company, we chose the latest one to two boards such that we tested the most recent iterations of the most supported board families for each company. From these selections, three to five boards were chosen for each company, amounting to 12 total boards under consideration. We then performed high-level research on the hardware capabilities, on-board peripherals, and feature specifications for each of these boards and recorded the information in a table so that we could compare them and determine which ones might provide the best fit for the course.

Once these high-level comparisons were complete, we chose the following four boards for our testing set: the Silicon Labs EFR xG26-DK2608A Dev Kit (EFR32), the STMicroelectronics (STM) NUCLEO-G491RE Dev Kit (STM32G4), the STM NUCLEO-H533RE Dev Kit (STM32H5), and the Espressif ESP32-C6-DevKitC-1 Dev Kit (ESP32C6). From a pre-testing point of view, each of these boards offered core embedded systems functionality like GPIO, ADC/DACs, and on-board peripherals, familiar development environments, and clear documentation that showed promise of fitting in well with an introductory embedded systems course. Some boards, like the EFR32 and the ESP32C6 were chosen for their lightweight and small form factorsWe evaluated these four boards against the current lab materials for the course and industry and educational standards to determine the ease with which the course labs could be outfitted to the board. so we could

decide for ourselves which one would be ideal for the ECE 2049 course. We investigated the EFR32xG26, the ESP32-C6, and both the STM32 NUCLEO-G491RE and the STM NUCLEO-H533RE. Although comparing a feature one board has that the other does not have may seem obvious, we need to remember that we are not necessarily trying to find the perfect board; we want the board that will be best for the course.

We had evaluation criteria for both the board and its IDE.

3.4 Objective 3: Modernize current lab demonstrations and assignments to align with the new lab kit

TODO

4 Preliminary Findings - Kyle

From these interviews, one professor mentioned that they favored the EFR32 xG24-DK2601B board after switching to it several years ago. The board follows a smaller form factor and ships with several sensors, enabling easy portability and innovation using the sensors. However, the professor mentioned that there is no textbook for many of the EFR32 family of microcontrollers, and that they created a booklet following chapters that generally explain the embedded systems software and hardware concepts directly relevant to class assignments to compensate. As we redesign the ECE 2049 course, this advice can be used to inform decisions on useful course material that can facilitate or supplement lecture and lab content, as well as gauge the ease of use that a board like the EFR32 with have at WPI.

In another interview, a professor from another school showed us the website they use for their course. Much like the custom booklet from the first professor, this website provides an outline of all the lectures and homework materials for the course, neatly packaged into a user-friendly and publicly accessible medium. Main topics are visible from the navigation bar on every subpage, and subtopics expand immediately upon opening a main topic page. The professor who created this page mentioned the importance of building a modular teaching medium, such that generic and constant topics within embedded systems (like communication protocols, interrupt handling, and ADC/DAC conversion, among others) are introduced and taught in a general format, with implementations on a specific board being introduced in their own subsections. This modularization maintains the constant portions of the course material while providing easily swappable sections that pertain specifically to the board, easing the process of upgrading the hardware and its connection to the course material in the future.

Furthermore, the same professor stressed the importance of teaching the fundamentals of interactions with embedded systems before introducing hardware abstraction layers. For example, students should know how to interact directly with memory, registers, and I/O via manual changes within the code before introducing hardware and software abstraction layers. This build paradigm follows a bottom-up approach to embedded system development, ensuring that building blocks for basic embedded development are established before quality-of-life features that abstract those concepts are allowed. This teaching style promises a complete understanding of the interactions between hardware and software, a quality that is especially important in an introductory embedded systems course. From the third professor we interviewed, the idea of having design components in the course was introduced. In their course, there is a large project involving the proper operation of a gas valve that spans the entire semester, where students use a set of peripherals learned from labs to contribute to their final projects. The course is structured such that every lab introduces a new concept, peripheral, or integrated circuit as a building block that can be used in the final project for the course. Students can choose the design that ultimately accomplishes the requirements of the final project. Through this course structure, students learn how to use peripherals in an embedded context and determine how to integrate them into a final project individually, teaching valuable design skills.

From all professors, the idea of providing lab kits to students was unanimous, citing ease of access for students, reduction of uncertainty in payment, and a more open selection of peripherals. Department-managed boards make course planning easier, according to one professor, minimizing the risk of students not being able to afford or obtain a board in time for the course. Professors and teaching assistants can verify the state of the boards before the course starts and put together a new kit or fix an existing one easily.

4.1 Board Testing - Shannon

We took a process-based approach to testing the final boards. We created two charts, one to evaluate the IDE criteria and one to evaluate the board itself. We assessed the features present in the IDE and the ease of use. The chart for the board included both qualitative and quantitative data, such as the number of connector pins and the quality of the interface with other peripherals. We also considered the quality of documentation and beginner-friendliness.

In order to fill out this chart, we created set goals derived from the current ECE 2049 labs. This included basic LED blinking, a Simon Says game, and a sensor implementation program that utilized the analog digital converter. We documented our progress as we attempted these labs, noting where things were difficult or straightforward. From this testing, we aimed to gather information on the following metrics for each board: ease of environment setup, quality of documentation, quality of community support, ease of interfacing with external peripherals, access to standard sets of expansion boards, and overall physical and programmatic board quality during development.

4.2 EFR xG26-DK2608A - Shannon

The biggest draws of the EFR were the AI/ML Hardware Accelerator and numerous on-board sensors. This board contained a sensor for temperature/humidity, inertia, pressure, ambient light, and magnetic fields. It had multiple microphones, 2 push buttons, a reset button, and a user-controlled RGB LED.

Silicon Labs created its own IDE called Simplicity Studio. However, further investigation revealed that while Simplicity Studio 5 is truly an IDE, the newest version, Simplicity Studio 6, is not. Version 6 is more like a shell to manage packages and stuff. All coding happens in Visual Studio Code. The user must also install XXX extensions for it to work in VSC.

Simplicity Studio offers over 900 demos, projects, and examples ready to launch when creating a new project. However, because of this, it was not easy to create a blank project. While investigating this, we accidentally discovered the solution; we needed to create an "empty" project.

Every single Simplicity Studio project, including the ones labeled empty or bare metal, would generate with folders of code. This was nice because ..... However, the main.c file was always created with a kernel and calls to functions, sl_main_init(), app_init(), sl_main_process_action() and app_process_action().

This set-up was very event-driven and thus not directly aligned with the teachings of the class.

In order to add functions, you had to search in the SS launcher. When searching for a ceratin function, multiple responses would appear. There were categories like Bluetooth, Zigbee, RTOS, and Third Party, Application, and Platform. With options in multiple categories with similar names, it could get confusing as to which one you actually need.

A beginner likely wouldn't know the difference between "IO Stream," "IO Stream - Recommended Stream for debug output," "IO Stream: Recommended Stream," "IO Stream: Recommended Stream for a console," "IO Stream: Retarget STDIO," or "IO Stream: STDLIB Configuration."

Having two user buttons on the board was very nice when doing the Simon Says project. We were able to run the full game, with a red LED flash requiring BTN0 to be clicked and a blue LED flash requiring BTN1 to be clicked, without any external components. Satisfied with this success, we changed the project name from "empty," which SS assigned upon creation when we chose the "Empty C Project" option, to simon_says. However, things started to fall apart as the project would no longer build. Further investigation revealed that the name given to the project when it is first generated is hard-coded into the configuration files. We were able to utilize the debugger to find at least two instances of this, but it still didn't work. We deduced that it must be hard-coded in more places and it wasn't practical to keep trying.

Overall, this board was simply not beginner-friendly enough for our purposes. All the sensors and the ML accelerator are cool and would be helpful for an older student who wanted to use a familiar platform for a different application. However, that is not the most important factor, and thus we took this board out of the running before our final conversation about evaluating the decision. (idk need to change this)

4.3 ESP32-C6-DevKitC-1 - Kyle

The next board we tested was the ESP32-C6-DevKitC-1 board, which has a large draw due to its simple setup and the ease with which a beginner embedded programmer can follow its guides and instructions. The ESP is a smaller board, both in size and the features that it offers. Its wireless radio capabilities through Wi-Fi, Bluetooth 5 (including Low Energy), Zigbee, and Thread, and its focus on integration into

IoT devices, make it a more modern, higher-level chip. Within the MCU on the DevKit, its built-in wireless capabilities are the centerpiece; yet, the chip also supports a wide range of peripherals that enable other external functionalities.

The MCU has GPIO support, with most GPIO pins bound to specific header pins on the DevKit. These header pins are often multiplexed with other functionalities and peripherals, due to the system's small form factor. On top of GPIO, the MCU offers SPI, parallel IO, UART, I2C, I2S, and TX/RX support for communication. Additionally, the chip supports specific LED PWM output and motor control PWM output for controlling externally connected peripherals. It contains two main hardware timers, a system timer, a watchdog timer for firmware wellness checking, and a single seven-channel ADC. Connections between the host machine and the board are made via one of two USB ports at a time: one supporting debugging via serial UART and the other supporting JTAG debugging. Finally, the board contains a temperature sensor accessible via the on-board ADC and an addressable on-board RGB LED.

Compared to the EFR32, the ESP32 board does not have many on-board sensors, relying upon externally connected sensors to implement the functionality. The vast range of functionality in the wireless capabilities of this board differentiates it from the others. For an IoT-connected device, the ESP32-C6 is a strong choice because of its support for different protocols and communication interfaces.

A notable aspect of this board is the extensive documentation available for various parts of the board. The primary form of documentation is user guides, including a general overview of the board and specific peripheral functionality, including how to use them. These user guides are website-based, using dynamic links, inline code examples, images, and outlines that simplify navigation. There are two main user guides: a hardware reference manual and a programming guide. The former describes the hardware layout of the specific DevKit, with descriptions of major board components like the MCU, pin headers, power connectivity, and USB ports, plus a full breakdown of the header pin layout including names, types, and multiplexed functionalities. The latter describes the exact steps to get started with programming the board. It is organized with the most basic steps first, like the IDE environment setup and SDK installation, before moving into more specific peripheral setup guides. The programming guide is extensive, with very specific and well-documented processes for setting up almost every peripheral on the board. Further, the technical reference manual, accessible via the programming guide, contains specific instructions for how to set specific fields in registers to enable certain peripherals and configure them, which is a skill that is necessary in any embedded system course. The tutorial-like nature of the technical reference manual would serve as an easily digestible introduction to understanding chip datasheets. This is a quality that makes this board strong.

The IDE experience is smooth. Installation on all supported operating systems is easy and well-documented in the Programming Guide. On Windows, a single installer is used to install the entire environment, including the ESP-IDF software, the SDK backend responsible for all development tooling on Espressif systems, including custom compilers and debuggers. On Linux and Mac, the IDE installation is separated from the ESP-IDF installation; however, a built-in tool can be used to automatically install the latest version of the ESP-IDF software on any supported operating system. The main installable IDE is based on a newer version of the Eclipse C/C++ Development Tooling (CDT) IDE, so it is a familiar choice for many. Programming with the IDE is straightforward, with responsive code IntelliSense and easy-to-understand macro names for common registers used with the board. The debugger integrates directly with the Eclipse CDT debugging GUI, including windows for examining expressions, peripheral memory-mapped registers, and the call stack. It can also be switched to step through the code by instruction instead of by line, allowing for a complete debugging experience. Compilation is always fast on macOS and Linux, but Windows requires the included Windows Defender Exclusion script to be run such that it does not reduce compiler performance.

For programming the board itself, there is always a real-time operating system (RTOS) running on the board. Empty projects always start with an RTOS (typically FreeRTOS), and the main entry point for the system is through a task callback function. As such, it is not possible to achieve full bare-metal programming on this board. This does provide an easy way of including specific drivers and libraries for peripherals. Espressif offers a tool known as the ESP Component Registry that allows one to install smaller ESP-specific libraries to support specific external peripherals into a project with a single button. Additionally, CMake options allow the loading of specific Espressif drivers that enable one to program specific on-board peripherals. By default, these drivers are not included, but when compiled into the program, their functions and macros become available to import and use.

Further, the SDK ships with a Hardware Abstraction Layer (HAL) that eases the configuration and

interaction with peripherals in code. However, the easy-to-use technical reference manual allows one to simply use peripheral registers to set them up instead of the HAL if desired. To configure specific hardware and software systems on the chip, a single file can be modified using the ESP-IDF tools to enable or disable certain features. This system, however, can be confusing to follow, as it appears as a large list of checkboxes that are not clearly organized.

The ESP32-C6 DevKit has 31 addressable GPIO pins multiplexed to soldered pin headers. However, we found that it was difficult to put the board onto a breadboard for development. We thought that the board might be too wide for the typical breadboard pins, so we tried to connect it to two breadboards separated down the middle; however, the board did not easily slot into the pin holes on the board. Additionally, there is no shield board support for these boards, including Arduino, STM, or QUIC header connections, meaning that all shield boards would need to be manually wired up to function correctly. Further, only two general-purpose timers are available on the board, which limits some functionality with more complex projects that may need to use more than two timers. Despite this limitation, timers are dynamically set up at runtime through the RTOS, as every timer is initialized when the RTOS starts, meaning setting up timers is a very simple process.

Finally, the ESP32 development environment has a Visual Studio Code (VSCode) extension that brings the Espressif-IDE functionality to VSCode. The extension is fully featured, with debugging, building and compiling, ESP-IDF installation and configuration, project configuration, and a clean SDK configuration GUI. The separation of all SDK features and toolchains into the ESP-IDF program allows both the Eclipse-based Espressif IDE and the VSCode extension to operate in almost equivalent manners, except for differences in GUIs. The VSCode environment is much more modern and, from our testing, more user-friendly.

4.4 STM32 NUCLEO - Kyle

For the STM32 Nucleo boards, we tested two versions from two different subfamilies, including the NUCLEO-G491RE and the NUCLEO-H533RE. Development on both boards was very similar, as they both use the same programs and IDEs. As such, we will evaluate the development environment jointly in this section, followed by an analysis of the boards' individual differences.

With every STM32 development board, a software stack consisting of three programs is used: the *STM32CubeMX* code generator program, the *STM32CubeIDE* development program, and the *STM32-CubeProg* board programmer program. The *STM32CubeMX* program contains the functionality and tooling for setting up all software and hardware resources to create a project, equivalent to the ESP-IDF tooling in the Espressif environment. It contains functionality to install SDK packages for specific STM boards as well as a simple wizard for creating a project for a specific board. The STM32 project creation environment works by setting up all software and hardware systems using a GUI-based setup system, which the CubeMX program uses to automatically generate the code that is necessary to set up the peripherals. As such, getting started with any peripheral on the system is easy, as much of the setup logic can be abstracted away under this setup process. The configuration screen also contains a GUI-based pinout diagram that allows graphical assignment of functions to multiplexed pins on the MCU, which correspond to specific pin headers on the Nucleo DevKit, making GPIO and peripheral outputs easy to understand and set up.

The CubeMX program configures the default compiler/linker to one of a set of IDE and toolchain development environments that the user chooses, with the two main choices being the STM32CubeIDE and CMake environments. The former sets up the project structure to integrate directly with the official, separately installed Eclipse CDT-based STM32 IDE, with functionality to open the project immediately in the program. The latter sets up the project to integrate with the STM32 VSCode extension. The code generator can then be run to set up the project into the correct configuration and IDE, where the project is functional immediately after code generation and can be easily built and developed in either the STM32CubeIDE or the VSCode extension.

4.4.1 STM32CubeIDE - Eclipse - Kyle

The STM32CubeIDE is based on the Eclipse CDT framework; however, it ships with an older version of that framework that does not include many usability and user experience changes of newer versions. The IDE does support installing extensions, although many use older versions to support the specific version of Eclipse CDT. The IDE does not ship with Git support by default, but it is easily installable through

the EGit extension in the Eclipse Marketplace subprogram. The overall functionality of the IDE is the same as the Espressif IDE, with direct compiler, IntelliSense, and debugging support through the Eclipse CDT framework. Since project creation and configuration are completely handled by the *STM32CubeMX* program, the project creation functionality is unused in the IDE.

Programming using the IDE is a relatively smooth process, given that IntelliSense (activated with the key combination "Ctrl + Space" on Windows and Linux or "CMD + Space" on Mac) helps autocomplete code, and the file structure is easy to follow on the left-hand side. The STM32 HAL is feature-rich, well-documented, and easy to use for changing register values and configuring peripherals correctly. However, just like the ESP32C6, this HAL can be bypassed, and registers can be altered directly. The STM32 Nucleo boards do not run an RTOS by default, running everything on a bare-metal program. One can optionally load an RTOS, like FreeRTOS, into the program if desired; however, it is not required for proper functionality.

Within each source code file, separate sections are provided for user code to be placed into. User code must be placed between these section markers so that code generation ignores them and does not overwrite them. If code is written outside of those sections, it will be irreversibly deleted when code generation is run. Further, no user code should be written in the automatically generated functions, as those will be overwritten as well.

Building and debugging code is simple in the IDE, as the STLink debugger also integrates directly with the Eclipse debugging framework. Much like the Espressif-IDE, there are windows to observe the program call stack, code expressions, CPU registers, and SFRs for memory-mapped registers. Additionally, the debugger can be changed to step by instruction instead of by line. This provides a fully featured debugging experience.

4.4.2 STM32CubeIDE - VSCode - Kyle

STMicroelectronics also offers its fully updated and supported VSCode extension for a more modern development experience. The extension is self-installable and prompts the user to install the necessary dependencies and toolchains. It comes prepackaged with a custom version of the clangd C language server that integrates with the custom build environment. The VSCode extension, like the Eclipse STM32CubeIDE, does not have project creation abilities, delegating that functionality to the STM32CubeMX program. Projects can be generated to work out-of-the-box with the VSCode environment by selecting the CMake IDE option with the ST Arm Clang option selected for the Default Compiler/Linker. When opening the project in the VSCode extension, it can be imported by the extension by selecting the pop-up that appears in VSCode, which will automatically configure the project and prompt the user for any more libraries or SDKs that are needed.

CMake is used as the default compiler for projects, setting up a custom build target that builds the project. The only supported operation for running and flashing to the board involves starting a debugger, meaning simply flashing to the board does not have a task. However, a custom task can be created that calls upon the STM32CubeProg application to upload the built binary to the board. This task can be created project-specific or as a user task, allowing it to apply to all projects.

The ST-Link debugger installed with the STM32 SDK and toolchains links directly with the VSCode debugging system. The STM32 extension creates a debugging configuration that sets up and launches the debugger with the correct settings automatically; it simply needs to be selected first. The debugger has the same functionality as the Eclipse IDE, including tracking the values of local variables, CPU registers, SFRs, the current call stack, available breakpoints, as well as a view into the fault state registers on the chip, which are helpful for more specific debugging. One can step through the code by instruction or simply observe the assembly instructions by right-clicking on the call stack and opening the disassembly view.

4.4.3 STM Nucleo-G491RE - Shannon

As per the Nucleo-G491xC and Nucleo-G491xE datasheet [14], the NUCLEO-G491RE development board is based on the STM32G491RE MCU, a member of ST's G4 series designed for high-performance mixed signal embedded applications. The Nucleo development kit features the standard Nucleo-64 form factor/connectivity. This includes Arduino Uno V3 compatibility and ST morpho headers for GPIO access. The board includes an integrated ST-Link debugger connected with a micro USB. This cable also supplies power to the board. The on-board MCU has 512 KB of flash memory and 96 KB of SRAM, along with an additional 16 KB of core-coupled memory (CCM). This is good for a wide range of applications from simple projects

to complex real-time control systems. The Arm Cortex-M4 core operates at up to 170 MHz and includes a floating-point unit (FPU) and DSP instructions for enhanced mathematical performance. The board also has 86 multiplexed GPIO pins, which allows flexible assignment across peripheral functions. Key peripherals include a 16-channel DMA controller, 15 timers (including one 32-bit timer, three advanced motor-control timers, and multiple general-purpose/basic timers), and 16 communication interfaces consisting of SPI, I2C, USART/UART, LPUART, FDCAN, SAI, and USB 2.0 Full-Speed. The MCU also utilizes three 12-bit ADCs, supporting up to 36 input channels with sampling rates up to 4 Msps, four 12-bit DAC channels, and four comparators. Additionally, it contains two accelerators, CORDIC and FMAC, for trigonometric and digital filtering operations.

4.4.4 STM Nucleo-H533RE - Kyle

As per the Nucleo-H533xx datasheet [15], the Nucleo-H533RE development board is based on the STM32-H533RE MCU, which is a member of STM's high-performance MCU family. The Nucleo development kit is mostly the same as the Nucleo-G491RE, except that the connection interface to the ST-Link debugger is via the newer USB-C instead of the Micro-USB connector. The main difference is in the on-board MCU, which offers different features than the G491RE. With 512 KB of flash memory and 272 KB of SRAM, the MCU can support a wide variety of simple to complex applications. It has 112 different IO options, with output pins being multiplexed to different peripherals, including GPIO. The chip includes memory security features and various cryptographic accelerators for use in the fields of cryptography. Some other peripherals include two DMA controllers, 16 different timers of varying resolution, 34 different communication interfaces, two 12-bit ADCs with multiple channels of input, one 12-bit DAC, and a digital temperature sensor.

4.5 Board Choice - Kyle

The testing performed on these four boards has given ample information on the metrics we aimed to test, mentioned above. As mentioned in the analysis of the first board from Silicon Labs, the overly high-level programming environment and discontinuity between the Silicon Labs Launcher and the IDE extension in VSCode made the board difficult to use and understand from a beginner's perspective. More time was spent resolving issues with the setup of the environment and the board's connectivity than was actually spent programming it. While the AI/ML features among the many different sensors offer an all-in-one solution for creativity on the board, the overhead of setting up the environment is not compatible with a seven-week term. Additionally, we are not confident in its ability to teach important low-level embedded systems concepts at the level required by this course due to its high-level program structure. For these reasons, this board will not be considered for this course.

The STMicroelectronics Nucleo-G491RE and Nucleo-H533RE, and the ESP32-C6-DevKitC-1 board from Espressif Systems remain as candidates for possible use in this course. Due to the similarities between the environments and programming process of the Nucleo boards, we will first evaluate both boards together with the board from Espressif Systems. Both boards contain very similar tooling and IDE structures, with project creation occurring through an external core tool that integrates with the IDE. In the ESP32 board, the ESP-IDF tool is directly integrated into the IDE, with project creation and SDK installation occurring within the IDE itself instead of needing a separate program. The STM32 boards separate this process, with separate apps responsible for project creation, SDK installation, and board configuration. Despite this separation, the integration between them is seamless and produces minimal to no issues between project creation, configuration, and user development. Both boards also have well-supported and up-to-date VS Code extensions that integrate with their respective core tools well and have easy setup processes. In both cases, debugging is easier on the VS Code extension than it is in the Eclipse version of the IDE, and the features offered on both are mostly equivalent.

In terms of the quality of documentation, both have extensive, clear, and easy-to-follow documents and website-based user guides. Both have extensive technical reference manuals and hardware datasheets that allow one to program individual components of the board easily. However, the ESP32 includes more tutorial-like concepts within its technical reference manual, including clear steps outlining which fields in which registers should be changed to which value to enable certain functionality. For individuals reading technical reference documentation for the first time, this can be very useful. While the STM technical reference manual does not have individual step-by-step instructions, it has more detailed descriptions on what each

register does and how to set them up, producing overall a more detailed documentation without step-by-step instructions. Additionally, community support for both boards and environments is strong, with similar IDE structures and tooling existing for many years. This means that users of both platforms will most likely be able to find a solution to issues via either official documentation or community notes.

For the ability to interface with external peripherals, the STM Nucleo boards are much better. Both Nucleo boards have a large surface area that allows them to sit flat on a surface when connected to the computer. The pinouts on the board have connections on top of and underneath the board for easy access, and the board natively supports the Arduino and the ST Morpho driver connection pinout standards. As such, driver boards adding extra external functionality from both Arduino and STMicroelectronics are supported natively. The ESP32 does not have this support. Its only pinouts are on the bottom of the board, meaning it can't sit flat on a surface during development, and makes physically connecting external devices awkward. The board does not support any standardized set of pinouts that work natively with driver boards, meaning connections to them require manual wiring for each connection. The board is intended to be used on a breadboard as well; however, the width of the pins is slightly too large to fit easily into the slots, even when freeing the range of motion by splitting the sides of the board between two breadboards. As such, damage to the board may be easily caused simply by attempting to plug it into the breadboard. During testing on both boards, connections were made to a custom breadboard setup with buttons and LEDs wired to separate GPIO pins. With the top-mounted pins on the Nucleo boards, the wiring process for that custom breadboard was cleaner and easier, and overall provided a much better experience.

Finally, physically, the STM32 Nucleo boards felt sturdier and more robust than the ESP32. The buttons have extenders on them to make them easier to push without getting your finger close to the electrical components and pins, and the board includes a user button to perform general-purpose actions. The ESP32 was more awkward to hold and set down on the table, especially when wires are connected to the board. Additionally, there is no user button on the ESP32, limiting out-of-the-box functionality on the board. From a software perspective, the ESP32 always requires a running RTOS, which creates problems with achieving true bare-metal embedded programming. This concept is a core feature of the ECE 2049 course, and allows students to learn a foundational level of how embedded systems work. The STM32 defaults to running on bare metal with a bypassable HAL and the option to enable an RTOS if needed. The user is not bound to using this RTOS and, as such, can learn about core embedded systems program functionality and structure easily. Further, this paradigm on the STM improves debugging since the system will not try to shift to the RTOS kernel or other tasks while the main task is running, producing expected and easily followable debugging paths for beginners.

Since the functionality of the programming environments for both families of boards is so similar, with almost equivalent versions of features between their official Eclipse-based IDEs and VS Code extensions, the physical and some of the programming limitations of the ESP32 compared with the STM32 Nucleo boards push it down the list of desirable boards for this course. For students getting started with the board, the traditional software structure of the STM32 Nucleo boards aligns better with the introductory concepts to embedded programming offered by the course, and the simplified hardware connections will make forming adaptive lab kits that can change over the course of many years much easier. Additionally, both Nucleo boards have a 10-year longevity commitment until 2035 from STMicroelectronics [16][17], ensuring that both boards will be available and in support for at least 10 years, offering plenty of time for the board to be integrated into the course and potentially upgraded again in the future.

This leaves the choice between each of the Nucleo boards. Both boards share many of the same peripherals with slightly different levels of performance, and both are the same price. But ultimately, the Nucleo-H533RE contains more upgraded sets of peripherals, with a greater number of GPIO outputs, a faster base MCU clock speed of 240 MHz, dedicated hardware modules to cryptography and software security, slightly more timers and communication interfaces, and the more modern USB-C connection interface to the ST-Link debugger and User USB connector. While the Nucleo-H533RE has fewer ADCs and DACs than the G491RE, the number of supported channels on both is still perfectly sufficient for the course. While both boards would be sufficient for teaching this course, the extra and more expansive features on the H533RE push it ahead of the G491RE, especially in the area of use after the class. With the ECE 2049 course, the board becomes a tool that students can apply to future academic or personal projects in the field of embedded computing. As such, the board with more features, including advanced ones that may not be used in the course, is helpful for students who choose to pursue embedded programming after taking the course. Additionally, the extra

features at the same price point make the H533RE an easy choice. As such, the final board choice for ECE 2049 will be the STM32 Nucleo-H533RE Development Kit.

References

[1] M. Jiménez, R. Palomera, and I. Couvertier, *Introduction to Embedded Systems: Using Microcontrollers and the MSP430*, en. New York, NY: Springer, 2014, ISBN: 978-1-4614-3142-8 978-1-4614-3143-5. DOI: 10.1007/978-1-4614-3143-5. Accessed: Oct. 14, 2025. [Online]. Available: https://link.springer.com/10.1007/978-1-4614-3143-5.

[2] C. Cai, "Theory And Application Analysis of Embedded Systems," en, *Highlights in Science Engineering and Technology*, vol. 71, pp. 171–176, Nov. 2023. DOI: 10.54097/hset.v71i.12688. [Online]. Available: https://www.researchgate.net/publication/376859370_Theory_And_Application_Analysis_of_Embedded_Systems.

[3] A. Barkalov, L. Titarenko, and M. Mazurkiewicz, *Foundations of Embedded Systems* (Studies in Systems, Decision and Control), en. Cham, Switzerland: Springer International Publishing, 2019, vol. 195, ISBN: 978-3-030-11960-7 978-3-030-11961-4. DOI: 10.1007/978-3-030-11961-4. Accessed: Nov. 3, 2025. [Online]. Available: http://link.springer.com/10.1007/978-3-030-11961-4.

[4] Grand View Research, "Embedded System Market (2025 - 2030)," en, Grand View Research, San Francisco, CA, Tech. Rep. 978-1-68038-129-0, p. 110. Accessed: Nov. 3, 2025. [Online]. Available: https://www.grandviewresearch.com/industry-analysis/embedded-system-market.

[5] M. Wu and X. Chen, "Application of Internet of Things and embedded technology in electronic communication," *Measurement: Sensors*, vol. 34, p. 101 246, Aug. 2024, ISSN: 2665-9174. DOI: 10.1016/j.measen.2024.101246. Accessed: Oct. 14, 2025. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S2665917424002228.

[6] M. Samiullah, M. Z. Irfan, and A. Rafique, *Microcontrollers: A Comprehensive Overview and Comparative Analysis of Diverse Types*, en, National University of Sciences and Technology, Sep. 2023. DOI: 10.31224/3228. Accessed: Sep. 22, 2025. [Online]. Available: https://engrxiv.org/preprint/view/3228.

[7] Espressive Systems, *ESP SoCs — Espressif Systems*, en. Accessed: Oct. 14, 2025. [Online]. Available: https://www.espressif.com/en/products/socs.

[8] STMicroelectronics, *STM32 Microcontrollers (MCUs)*, en. Accessed: Oct. 15, 2025. [Online]. Available: https://www.st.com/en/microcontrollers-microprocessors/stm32-32-bit-arm-cortex-mcus.html.

[9] STMicroelectronics, *STM32 Nucleo-64 boards*, Jan. 2025. Accessed: Oct. 14, 2025. [Online]. Available: https://www.st.com/resource/en/data_brief/nucleo-c031c6.pdf.

[10] I. Ibrahim, R. Ali, M. Zulkefli, and N. Elfadil, "Embedded systems pedagogical issue: Teaching approaches, students readiness, and design challenges," *American Journal of Embedded Systems and Applications*, vol. 3, no. 1, pp. 1–10, Mar. 2015, ISSN: 2376-6085. DOI: 10.11648/j.ajesa.20150301.11. Accessed: Oct. 13, 2025. [Online]. Available: https://www.researchgate.net/publication/274303511_Embedded_Systems_Pedagogical_Issue_Teaching_Approaches_Students_Readiness_and_Design_Challenges.

[11] Joint Task Group on Computer Engineering Curricula, "Curriculum Guidelines for Undergraduate Degree Programs in Computer Engineering," en, Association for Computing Machinery and IEEE Computer Society, Tech. Rep. CE2016, Dec. 2016, p. 149. Accessed: Oct. 24, 2025. [Online]. Available: https://www.acm.org/binaries/content/assets/education/ce2016-final-report.pdf#page=11.10.

[12] Texas Instruments, *MSP430F552x, MSP430F551x Mixed-Signal Microcontrollers*, en, Dallas, TX, Sep. 2020. Accessed: Oct. 15, 2025. [Online]. Available: https://www.ti.com/lit/ds/symlink/msp430f5529.pdf?ts=1760506579932&ref_url=https%253A%252F%252Fwww.ti.com%252Fproduct%252FMSP430F5529.

[13] IPEDS, *About IPEDS*, en. Accessed: Oct. 14, 2025. [Online]. Available: https://nces.ed.gov/ipeds/about-ipeds.

[14] STMicroelectronics, *STM32G491xC STM32G491xE Datasheet - Production Data*, en-US, Apr. 2024. Accessed: Dec. 13, 2025. [Online]. Available: https://www.st.com/resource/en/datasheet/stm32g491re.pdf.

[15] STMicroelectronics, *STM32H533xx Datasheet - Production Data*, en-US, Nov. 2024. Accessed: Dec. 13, 2025. [Online]. Available: https://www.st.com/resource/en/datasheet/stm32h533re.pdf.

[16] STMicroelectronics, *STM32H533RE*, en. Accessed: Dec. 14, 2025. [Online]. Available: https://www.st.com/en/microcontrollers-microprocessors/stm32h533re.html.
[17] STMicroelectronics, *STM32G491RE*, en. Accessed: Dec. 14, 2025. [Online]. Available: https://www.st.com/en/microcontrollers-microprocessors/stm32g491re.html.